Name: _____ ID: _____

EXAM 1 - VERSION B/2
Fall 2026
ACC 380K
Coding and Data Analysis in Accounting

Please read all instructions carefully:

1. This exam has 25 questions, 1 point each, 25 points total.
2. To ensure fairness, please do not open this exam until instructed to begin.
3. Please write your name and student ID: (1) on the back of ALL pages of the provided bubble sheet, in the space provided on the front of the first page of bubble sheet, and in the space provided at the top of this cover page of the exam.
4. This is an "open-book exam". You may access any *written* notes that you have brought with you to the exam. However, you may not access any technology other than a calculator (i.e., no phones, laptops, tablets, etc.).
5. Evidence of scholastic dishonesty can result in a zero for this exam and/or a failing grade for the course. Scholastic dishonesty includes:
   - Any sharing of information about this exam with other students. This includes all communications between students taking the exam in a different section, evidence of which will result in appropriate disciplinary action.
   - Accessing any technology during the exam, other than a calculator.
   - Viewing any other student's exam.
6. For each multiple choice question, fill in the bubble on your answer sheet that corresponds with the best answer. Answers marked in your test booklet will not be counted!
7. Please ensure that your exam contains all 13 pages, including this page.

Good luck. You have one hour and 15 minutes to complete the exam.

1. The code below is meant to count annual reports (10-K), including amendments, whatever the capitalization:

```
forms = ["10-K", "10-k/a", "8-K", "10-Q", "10-K405"]
count = 0
for f in forms:
    if "10-K" in f.upper():
        count = count + 1
print(count)
```

What is printed?

(a) `5`

(b) `2`

(c) `4`

(d) `3`

(e) `1`

2. This loop attempts to count appearances of a phrase without using `.count()`:

```
text = "fee fee fee"
pos = text.find("fee")
count = 0
while pos != -1:
    count = count + 1
    pos = text.find("fee", pos + 1)
print(count)
```

What is printed?

(a) `2`

(b) `The loop never stops.`

(c) `1`

(d) `4`

(e) `3`

3. Consider the following code:

```
balances = [500, 320, -20, 400, -75]
for i in range(len(balances)):
    if balances[i] < 0:
        print("first overdraft at position", i)
        break
```

What is printed?

(a) `first overdraft at position -20`

(b) `first overdraft at position 2`
`first overdraft at position 4`

(c) `first overdraft at position 4`

(d) `first overdraft at position 2`

(e) `first overdraft at position 3`

4. Consider the following code:

```
def discount(price, pct):
    return price * pct

def sale_price(price):
    return price - discount(price, 0.25)

print(sale_price(80))
```

What is printed?

(a) `79.75`

(b) `None`

(c) `60.0`

(d) `20.0`

(e) `80`

5. Invoices are matched to an approved-vendor list. Vendor 9 is not approved, and vendor 10 has no invoices.
The DataFrame `invoices`:

	invoice	vendor_id	amount
0	A1	7	1200
1	A2	7	450
2	A3	8	980
3	A4	9	3000

The DataFrame `vendors`:

	vendor_id	vendor
0	7	Acme
1	8	Brio
2	10	Cato

How many rows does `pd.merge(invoices, vendors, on="vendor_id", how=H)` return for `H` = "left", "inner", and "outer", respectively?

(a) 4, 3, 4

(b) 4, 2, 4

(c) 5, 3, 4

(d) 4, 3, 5

(e) 3, 3, 5

6. Consider the following code:

```
inv = pd.DataFrame({"vendor": ["Acme", "Brio", "Acme", "Cato", "Brio"],
                    "amount": [1200, 300, 4500, 800, 2600],
                    "paid": [True, False, False, True, False]})
a = inv.loc[(inv["amount"] > 1000) & (inv["paid"] == False)]
b = inv.loc[(inv["amount"] > 1000) | (inv["paid"] == False)]
print(len(a), len(b))
```

What is printed?

(a) `3 3`

(b) `ValueError: The truth value of a Series is ambiguous`

(c) `2 4`

(d) `2 5`

(e) `4 2`

7. A balance of $100 is paid down by $30 per day:

```
remaining = 100
days = 0
while remaining > 0:
    remaining = remaining - 30
    days = days + 1
print(days, remaining)
```

What is printed?

(a) `3 10`

(b) `3 -20`

(c) `4 -20`

(d) `The loop never stops.`

(e) `4 0`

8. Consider the following code:

```
forms = {"SBUX": ["10-K", "8-K"], "TSLA": ["10-Q"]}
forms["TSLA"].append("8-K")
print(len(forms), len(forms["TSLA"]), forms["SBUX"][-1])
```

What is printed?

(a) `2 2 8-K`

(b) `2 2 10-K`

(c) `3 3 8-K`

(d) `3 2 8-K`

(e) `2 1 8-K`

9. The function below labels invoice amounts:

```
def tier(amount):
    if amount > 1000:
        return "high"
    elif amount > 5000:
        return "very high"
    else:
        return "low"

print(tier(8000), tier(1000))
```

What is printed?

(a) `high low`

(b) `very high high`

(c) `high None`

(d) `very high low`

(e) `high high`

10. Consider the following code:

```
tickers = ["SBUX", "AAPL", "SBUX"]
ciks = [829224, 320193, 829224]
lookup = {}
for i in range(len(tickers)):
    lookup[tickers[i]] = ciks[i]
print(len(lookup), lookup["SBUX"])
```

What is printed?

(a) `KeyError: 'SBUX'`

(b) `3 829224`

(c) `3 [829224, 829224]`

(d) `2 [829224, 829224]`

(e) `2 829224`

11. Consider the following code:

```
amounts = [250, -40, 1200, -15, 80]
flagged = []
for a in amounts:
    if a < 0 or a > 1000:
        flagged.append(a)
print(len(flagged), flagged[-1])
```

What is printed?

(a) `5 80`

(b) `3 1200`

(c) `2 -15`

(d) `3 -15`

(e) `3 80`

12. As in Homework 3, the code slices a passage around a search phrase:

```
text = "Revenue grew while costs fell"
pos = text.find("costs")
print(text[pos - 6:pos + 5])
```

What is printed?

(a) `ile costs f`

(b) `e costs fel`

(c) `costs fell`

(d) `while costs`

(e) `while cost`

13. As in Homework 3's `contexts_by_year`, each year maps to a dictionary of numbered passages:

```
results = {2021: {1: "a", 2: "b"}, 2022: {1: "c"}}
total = 0
for year in results:
    total = total + len(results[year])
print(total)
```

What is printed?

(a) `5`

(b) `3`

(c) `2`

(d) `KeyError: 1`

(e) `4`

14. Consider the following code:

```
def label(company, form):
    return f"{company}: {form}"

for form in ["10-K", "8-K"]:
    print(label("SBUX", form))
```

What is printed?

(a)
```
{company}: {form}
{company}: {form}
```

(b)
```
SBUX: form
SBUX: form
```

(c)
```
SBUX: 8-K
```

(d)
```
SBUX: 10-K
SBUX: 8-K
```

(e)
```
company: 10-K
company: 8-K
```

15. The DataFrame `visits` below has one row per store per week (`week` is a date column). Consider the following code:

	PERSISTENT_ID_STORE	REGION	week	VISIT_COUNTS
0	11	CA	2020-03-02	3000
1	12	CA	2020-03-02	2000
2	13	CA	2020-03-02	1000
3	11	CA	2020-03-09	2600
4	12	CA	2020-03-09	1800
5	13	CA	2020-03-09	800
6	21	IA	2020-03-02	5200
7	21	IA	2020-03-09	4800

```
a = visits["REGION"]
b = visits[["REGION", "VISIT_COUNTS"]]
c = visits.loc[visits["REGION"] == "IA", "VISIT_COUNTS"]
```

What type of object is each of `a`, `b`, and `c`, respectively?

(a) Series, DataFrame, Series

(b) DataFrame, DataFrame, DataFrame

(c) Series, Series, Series

(d) Series, DataFrame, DataFrame

(e) DataFrame, DataFrame, Series

16. Using `visits` (previous question), the code below is meant to give the total visits in each week, for all stores together, as an ordinary two-column table:

```
weekly = visits.groupby("week")["VISIT_COUNTS"].sum()
weekly.loc[weekly["week"] >= "2020-03-09"]
```

The second line raises `KeyError: 'week'`. Why?

(a) The `groupby` should use `.mean()`, since `.sum()` removes the key.

(b) `weekly["week"]` needs double brackets, `weekly[["week"]]`, to work as a mask.

(c) After `groupby`, `week` is the index (row labels), not a column; adding `.reset_index()` to the first line fixes it.

(d) `.sum()` drops the `week` column because dates cannot be added.

(e) `.loc` cannot take a mask; use `.iloc` instead.

17. Consider the following code:

```
def add_tax(df):
    df["tax"] = df["amount"] * 0.1

inv = pd.DataFrame({"amount": [100, 250]})
result = add_tax(inv)
print(result, "tax" in inv.columns)
```

What is printed?

(a) `NameError: name 'df' is not defined`

(b) `None True`

(c) `None False`

(d) `inv True`

(e) `tax True`

18. The code below is meant to print the fiscal years 2021 through 2024:

```
year = 2021
while year <= 2024:
    print(year)
year = year + 1
```

What happens when the cell runs?

(a) It prints 2021, 2022, 2023, 2024, 2025.

(b) It prints 2021, 2022, 2023, 2024.

(c) It prints nothing.

(d) It raises an IndentationError.

(e) It prints 2021 over and over and never stops.

19. Consider the following code:

```
dates = ["2024-11-03", "2025-01-15", "2025-02-20", "2024-12-30"]
inv = pd.DataFrame({"date": pd.to_datetime(dates),
                    "amount": [100, 250, 50, 400]})
inv["year"] = inv["date"].dt.year
totals = inv.groupby("year")["amount"].sum().reset_index()
by_year = totals.sort_values("amount", ascending=False)
print(by_year.iloc[0]["year"], by_year.shape)
```

What is printed?

(a) `2025 (2, 2)`

(b) `2024 (2, 2)`

(c) `2024 (4, 3)`

(d) `2025 (2, 1)`

(e) `2024 (2, 1)`

20. Consider this version of Homework 3's `collect_contexts`:

```
def collect_contexts(text, search_phrase, context_length):
    contexts = {}
    appearance_count = text.count(search_phrase)
    search_start = 0
    for appearance in range(1, appearance_count + 1):
        position = text.find(search_phrase, search_start)
        start = position - context_length
        stop = position + len(search_phrase) + context_length
        contexts[appearance] = text[start:stop]
        search_start = position + len(search_phrase)
    return contexts

print(collect_contexts("Musk said Musk", "Musk", 3))
```

What is printed?

(a) `{0: '', 1: 'id Musk'}`

(b) `IndexError: string index out of range`

(c) `{1: '', 2: 'id Musk'}`

(d) `{1: 'Musk sa', 2: 'id Musk'}`

(e) `{1: 'Musk sa'}`

21. The function below is meant to report whether any invoice is over the approval limit:

```
def any_over(amounts, limit):
    for a in amounts:
        if a > limit:
            return True
        else:
            return False

print(any_over([200, 900, 1500], 1000))
```

What is printed?

(a) `[False, False, True]`

(b) `True`
`False`

(c) `True`

(d) `False`

(e) `None`

22. Consider the following code:

```
text = "risk: tariff risk and FX risk"
print(text.count("risk"))
print(text.find("risk", 5))
```

What is printed?

(a) 2
13

(b) 3
14

(c) 3
0

(d) 2
6

(e) 3
13

23. As in Notebook 8, an outer loop visits years and an inner loop visits quarters:

```
years = [2023, 2024]
quarters = [1, 2, 3, 4]
count = 0
for y in years:
    for qtr in quarters:
        if qtr % 2 == 0:
            count = count + 1
print(count)
```

What is printed?

(a) 8

(b) 2

(c) 4

(d) 1

(e) 6

24. Consider the following code:

```
def add_fee(charges):
    charges.append(25)
    return len(charges)

bill = [100, 40]
n = add_fee(bill)
print(n, bill)
```

What is printed?

(a) `2 [100, 40, 25]`

(b) `3 [100, 40]`

(c) `3 [100, 40, 25]`

(d) `None [100, 40, 25]`

(e) `2 [100, 40]`

25. The DataFrame `filings` and the output of `filings.dtypes` are shown below. The code is meant to keep every Starbucks (CIK 829224) filing:

	CIK	Company Name	Form Type	Date Filed
0	829224	STARBUCKS CORP	8-K	2025-11-04
1	829224	STARBUCKS CORP	10-K	2025-11-14
2	320193	Apple Inc.	10-K	2025-10-31
3	829224	STARBUCKS CORP	8-K/A	2025-11-20
4	1318605	Tesla, Inc.	8-K	2025-10-22
5	1318605	Tesla, Inc.	8-K/A	2025-10-29

`filings.dtypes`:

	dtype
CIK	int64
Company Name	object
Form Type	object
Date Filed	object

```
sbux = filings.loc[filings["CIK"] == "829224"]
print(len(sbux))
```

It prints `0`. Which statement is correct?

(a) `.loc` cannot take a mask; `filings["CIK"] == 829224` alone would return the 3 rows.

(b) `CIK` is text (`object`), so the comparison needs `.str` before `==` can match any rows.

(c) `.loc` matches the index labels, and no row has the index label "829224".

(d) `CIK` is `int64`, so comparing it with the text "829224" matches no rows.

(e) More than one of the other options is correct.